

## Arquivo: storage\_node.py

```
"""
DESCRIÇÃO GERAL:
Este módulo representa o servidor de um nó de armazenamento.
Cada nó roda de forma independente, escutando em uma porta própria e usando seu
próprio diretório físico para persistência.
"""

import argparse
import socket

from dfs.cluster.node_registry import NodeRegistry
from dfs.frame import recv_frame, send_frame
from dfs.storage.local_storage import LocalStorage
from dfs.application.node_service import NodeService

def build_parser() -> argparse.ArgumentParser:
    """
    Monta os argumentos da linha de comando do nó.
    """
    parser = argparse.ArgumentParser(prog="dfs-node")
    parser.add_argument("--node-id", required=True, help="identificador do nó")
    return parser

def main(argv=None) -> None:
    """
    Sobe um nó de armazenamento individual.
    """
    # Lê o identificador do nó.
    args = build_parser().parse_args(argv)

    # Consulta o cadastro central dos nós.
    registry = NodeRegistry()
    node = registry.get(args.node_id)

    # O shard do nó pode ser derivado da ordem do registry.
    shard_id = registry.index_of(node.node_id)

    # Cada nó usa sua própria pasta física.
    storage = LocalStorage(root=node.storage_dir)

    # Cria a lógica local do nó.
    service = NodeService(storage=storage, node_id=node.node_id, shard_id=shard_id)

    # Socket TCP do nó.
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
        # Permite reinício rápido do processo.
        server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

        # Liga o nó à sua porta específica.
```

```
server.bind((node.host, node.port))

# Coloca o nó em modo de escuta.
server.listen(5)

print(f"Nó {node.node_id} ouvindo em {node.host}:{node.port}")

try:
    while True:
        # Aguarda conexões enviadas pelo coordenador.
        conn, addr = server.accept()
        print(f"[{node.node_id}] conexão recebida de {addr}")

        with conn:
            # Lê a requisição.
            raw_request = recv_frame(conn)

            # Processa localmente.
            raw_response = service.dispatch(raw_request)

            # Devolve a resposta.
            send_frame(conn, raw_response)

except KeyboardInterrupt:
    # Encerramento manual do nó.
    print(f"\nNó {node.node_id} encerrado pelo usuário.")

if __name__ == "__main__":
    main()
```